

# Kết quả học tập theo tháng

💡 Validate trong code là chưa đủ — có những ràng buộc chỉ DB mới đảm bảo được.



### Mục tiêu buổi học

- Tạo bản ghi LearningResult theo từng tháng
- Composite unique key: student + class + month
- Hiểu tại sao validate bằng if/else trong service là chưa đủ



### Kết quả mong đợi

- POST learning-result thành công, POST trùng ra lỗi rõ ràng
- Giải thích được race condition và tại sao cần DB constraint
- Debug được lỗi duplicate và lỗi kiểu dữ liệu score

## Buổi 9 · Bạn đã biết điều này — nhưng hôm nay khác ở đâu?



### Validate ở trường: dùng if/else

- Kiểm tra trùng bằng: `if (repo.exists()) throw ...`
- Chạy đúng khi 1 người dùng
- 2 request cùng lúc: cả 2 check → cả 2 thấy "chưa có" → cả 2 insert → TRÙNG
- Lỗi này rất hiếm khi test một mình, chỉ lộ ở production



### Spring Boot thực tế: DB constraint

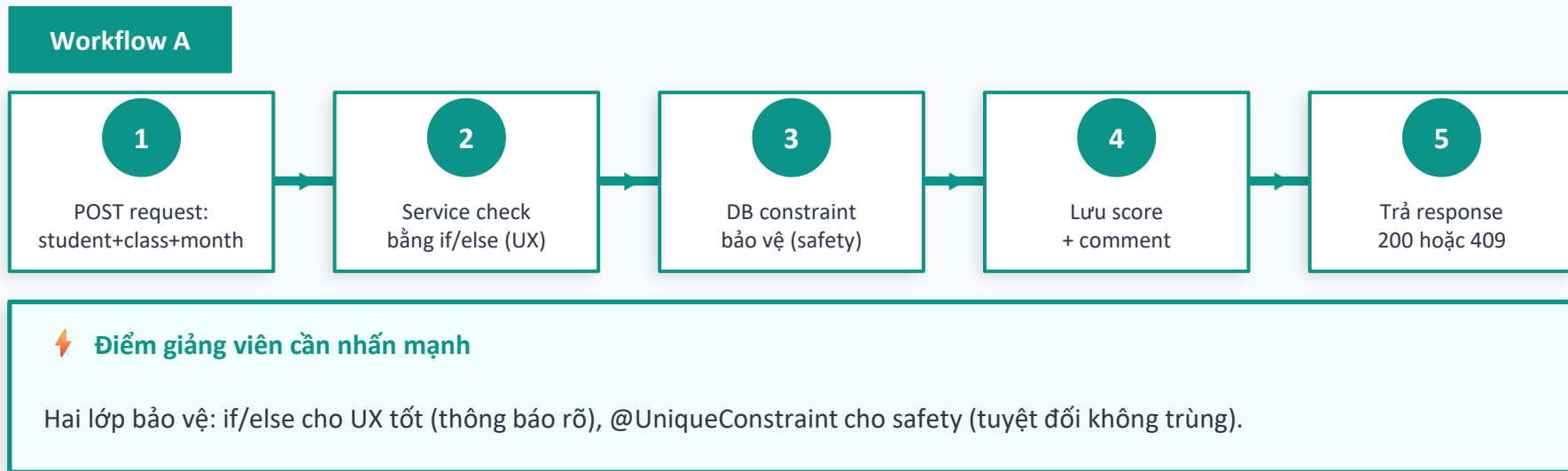
- `@UniqueConstraint(columnNames={"studentId","classId","month"})`
- Dù 1000 request cùng lúc: DB chỉ cho 1 cái thắng
- Cái còn lại nhận `DataIntegrityViolationException` → bắt và trả 409
- Validate bằng code để UX tốt, constraint để an toàn tuyệt đối

⚡ Điểm mới của buổi này: race condition — vì sao code đúng logic vẫn có thể sinh dữ liệu trùng.

## Buổi 9 · Mở file nào trước?

| # | Tên File                                | Mục đích khi mở                    | Điểm cần nói với SV                                                      |
|---|-----------------------------------------|------------------------------------|--------------------------------------------------------------------------|
| 1 | <b>LearningResult.java</b>              | Entity đánh giá định kỳ theo tháng | <i>Chỉ ra @UniqueConstraint — đây là điểm khác với bảng thông thường</i> |
| 2 | <b>LearningResultCreateRequest.java</b> | DTO input: score + comment + month | <i>Hỏi SV: nên dùng Integer hay BigDecimal cho score?</i>                |
| 3 | <b>LearningResultService.java</b>       | Logic validate + lưu kết quả       | <i>Chỉ ra chỗ catch DataIntegrityViolationException</i>                  |
| 4 | <b>LearningResultController.java</b>    | API tạo và xem kết quả             | <i>Controller trả 409 khi trùng — hỏi SV: 409 nghĩa là gì?</i>           |
| 5 | <b>LearningResultRepository.java</b>    | Truy vấn theo student / month      | <i>Xem method findByStudentIdOrderByMonthAsc — tại sao cần sort?</i>     |

## Buổi 9 · Workflow A — Nhập kết quả tháng



## Buổi 9 · Race Condition — lỗi chỉ lộ ở production

Validate bằng if/else hoạt động tốt khi test một mình. Nhưng ở production với nhiều user đồng thời...

### Request A (Giáo vụ Nguyễn)

repo.exists(month)? → false ✓

// Cả 2 thấy "chưa có"

repo.save(result) ✓

### Request B (Giáo vụ Trần)

repo.exists(month)? → false ✓

// Cả 2 thấy "chưa có"

repo.save(result) ✓

→ DB có 2 bản ghi TRÙNG! ✗

✓ Giải pháp: @UniqueConstraint ở tầng DB — dù 1000 request cùng lúc, chỉ 1 cái thắng, còn lại nhận exception.

## Buổi 9 · Workflow B — Xem tiến độ học tập

### Workflow B



### ⚡ Điểm cần nhấn mạnh

Sort theo month là bắt buộc — nếu không sort, phụ huynh thấy kết quả ngẫu nhiên theo insertion order.

## Buổi 9 · Lỗi thực tế & cách debug

Đây là những lỗi thường gặp — biết trước để không mất cả tiếng ngồi Google.

### DataIntegrityViolationException (409)

 **Nguyên nhân:** POST kết quả tháng mà tháng đó đã có bản ghi cho student+class đó rồi

 **Cách fix:** Bắt exception trong service, trả về 409 Conflict với message rõ ràng cho FE xử lý

### Score sai kết quả sau khi tính toán

 **Nguyên nhân:** Dùng Integer cho score thang 100 — chia / nhân mất phần thập phân

 **Cách fix:** Dùng BigDecimal hoặc Double, tùy thang điểm (10.0 hay 100.0) — quyết định sớm, viết vào entity

### Kết quả trả về không đúng thứ tự thời gian

 **Nguyên nhân:** Không có ORDER BY trong query — JPA trả dữ liệu theo insertion order

 **Cách fix:** Dùng findByIdOrderByMonthAsc() hoặc thêm @Query với ORDER BY month ASC

## Buổi 9 · Những phần code quan trọng

Tập trung đúng entity, service, controller cần dùng.

### Entity / Enum

- LearningResult
- @UniqueConstraint(  
• student+class+month)

→ Mô tả cấu trúc dữ liệu lưu vào DB

### Service

- LearningResultService
- create(): check + save
- findByStudent(): sort
- catch DataIntegrity → 409

→ Giữ toàn bộ logic nghiệp vụ

### Controller

- LearningResultController
- POST /api/learning-results
- GET /api/learning-results
- /student/{studentId}

→ Chỉ nhận request & gọi service

📌 Mở LearningResult.java → chỉ cho SV thấy @UniqueConstraint và giải thích sự khác biệt với validate trong code.



## Buổi 9 · Chuỗi endpoint cần viết

POST

/api/learning-results

*tạo kết quả tháng*

GET

/api/learning-results/student/{studentId}

*xem lịch sử*



### Trình tự demo gợi ý

- POST thành công lần 1 → kiểm tra DB có đúng 1 bản ghi
- POST trùng tháng → phải ra 409, message rõ ràng
- GET → list phải sắp xếp đúng thứ tự tháng tăng dần



### Điểm cần quan sát

- @UniqueConstraint có tạo index trong DB không? (Có — kiểm tra bằng SHOW INDEXES)
- Nếu score = 9.5 lưu là Integer, mất gì? (mất .5 — trở thành 9)
- Response DTO và Entity có phải giống nhau không? Tại sao không?



### Phải hoàn thành trong buổi

1. @UniqueConstraint rõ ràng: student + class + month
2. Bắt DataIntegrityViolationException → trả 409 (không để 500 ra ngoài)
3. Kiểu dữ liệu score phù hợp với nghiệp vụ (BigDecimal nếu có thập phân)
4. Sort theo month khi GET — không để FE phải tự sort

# 9

## Bài tập cuối buổi & Đầu ra



### Bài tập giao cuối buổi

- Tự động tính xếp loại từ score: Giỏi  $\geq 8.5$  / Khá  $\geq 7$  / TB  $\geq 5$  / Yếu  $< 5$
- Thêm filter theo month và year vào GET endpoint (query param)
- Tạo seed data 3 tháng liên tiếp — sau đó thử POST trùng, quan sát 409



### Sinh viên cần

- Giải thích được race condition và tại sao if/else chưa đủ
- Bắt và xử lý được `DataIntegrityViolationException` đúng cách
- Chọn được kiểu dữ liệu phù hợp cho score và giải thích lý do

*"Code đúng logic chưa đủ — DB constraint mới là lưới an toàn cuối cùng."*